Islamic University

Department of
Information and Communication Technology

"LAB REPORT"

Course Code: ICT-4102
Course Name: Information Coding and Transmission Laboratory

Submitted to:
Kazi Mowdud Ahmed
Assistant Professor
Department of Information and Communication Technology
Islamic University, Bangladesh

Submitted by:

Name: S. M. Sajjad Hossain Soykot
Roll: 2018040
Reg. No: 1315
Session: 2020-21

INDEX

Exp No.	Experiment Title
1	To develop a program to simulate and analyze the characteristics of common communication noises
2	To develop a program to analyze the characteristics of random noise under different mean and variance values
3	To develop a program to simulate a noiseless binary channel and analyze its data transmission performance
4	To develop a program to calculate the channel capacity of a Binary Symmetric Channel (BSC)
5	To develop a program for implementing the Shannon–Fano coding algorithm
6	To develop a program for implementing the Huffman coding algorithm for data compression

Experiment 1: To develop a program to simulate and analyze the characteristics of common communication noises.

Objective:

To write a program that analyzes the effect of communication noise, specifically focusing on **Gaussian noise**, **Impulse noise**, and **White noise**, and understand how they affect the quality of transmitted signals.

Theory:

In communication systems, noise refers to any unwanted signal that interferes with the transmission of data. Common types of noise include:

- **Additive White Gaussian Noise (AWGN):** A widely used noise model in communication systems. AWGN represents real-world channel noise.
    - Additive: noise is directly added to the signal.
    - White: constant power across all frequencies.
    - Gaussian: amplitude follows a normal distribution.
- **Impulse Noise**: Sudden spikes or impulses that can significantly degrade signal quality, often due to faulty equipment or external interferences.
- **Thermal Noise**: Noise generated due to the random motion of electrons in conductors. It is always present in electronic systems and is usually modeled as Gaussian noise.
- **Random Noise**: A type of noise that consists of unpredictable fluctuations and can vary in distribution and intensity.

Noise generally degrades the quality of a transmitted signal. The **Signal-to-Noise Ratio (SNR)** is a key parameter used to measure how much noise is present in a signal. Higher SNR means better signal quality.

$$\text{SNR (dB)} = 10log_{10}\left(\frac{P_{signal}}{P_{noise}}\right)$$

Algorithm:

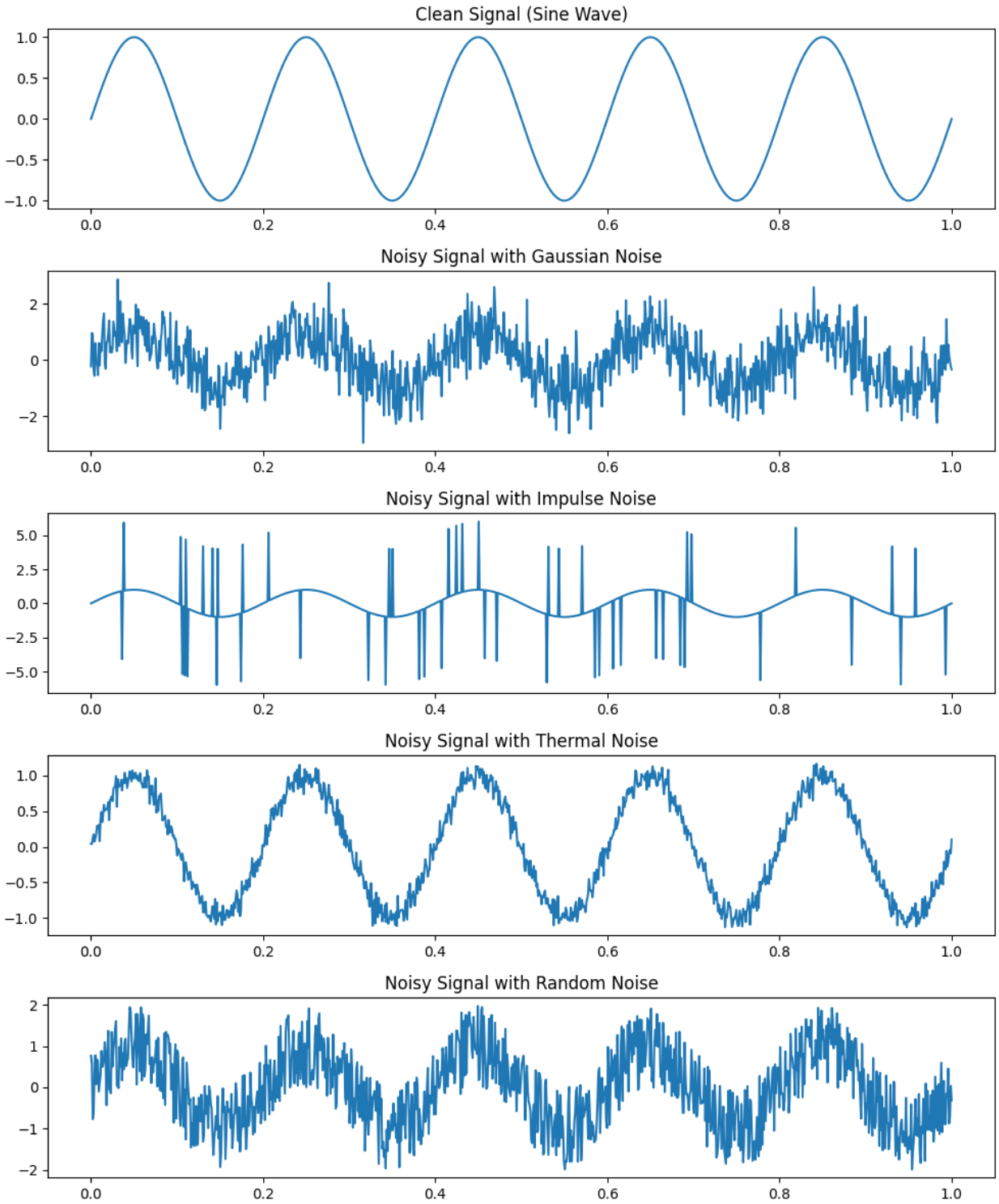
1. **Generate a clean signal**: A simple sine wave is chosen as the clean signal.
2. **Add AWGN noise**: Introduce AWGN noise based on a desired SNR.
3. **Add Impulse noise**: Introduce random impulses at random intervals in the signal.
4. **Add Thermal noise**: Introduce thermal noise using a low-variance Gaussian distribution.
5. **Add Random noise**: Introduce random noise using uniformly distributed random values.
6. **Analyze the noisy signal**: Visualize the clean and noisy signals to observe the effects of each type of noise.
7. **Compute Signal-to-Noise Ratio (SNR)**: Calculate the SNR for each noise type to quantify the impact of noise.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Simulate a clean signal (sine wave)
fs = 1000
t = np.linspace(0, 1, fs)
freq = 5
clean_signal = np.sin(2 * np.pi * freq * t)

# 1. AWGN Noise (Modified)
snr_db = 10  # Desired SNR in dB
signal_power = np.mean(clean_signal**2)
snr_linear = 10**(snr_db / 10)
noise_power = signal_power / snr_linear

awgn_noise = np.random.normal(0, np.sqrt(noise_power), fs)
noisy_signal_awgn = clean_signal + awgn_noise

# 2. Impulse Noise
impulse_noise = np.zeros(fs)
impulse_positions = np.random.randint(0, fs, size=50)
impulse_noise[impulse_positions] = np.random.choice([-5, 5], size=50)
noisy_signal_impulse = clean_signal + impulse_noise

# 3. Thermal Noise
thermal_noise = np.random.normal(0, 0.1, fs)
noisy_signal_thermal = clean_signal + thermal_noise

# 4. Random Noise
random_noise = np.random.uniform(-1, 1, fs)
noisy_signal_random = clean_signal + random_noise

# Plot
plt.figure(figsize=(10, 12))

plt.subplot(5, 1, 1)
plt.plot(t, clean_signal)
plt.title("Clean Signal (Sine Wave)")

plt.subplot(5, 1, 2)
plt.plot(t, noisy_signal_awgn)
plt.title("Noisy Signal with AWGN")

plt.subplot(5, 1, 3)
plt.plot(t, noisy_signal_impulse)
```

```
plt.title("Noisy Signal with Impulse Noise")

plt.subplot(5, 1, 4)
plt.plot(t, noisy_signal_thermal)
plt.title("Noisy Signal with Thermal Noise")

plt.subplot(5, 1, 5)
plt.plot(t, noisy_signal_random)
plt.title("Noisy Signal with Random Noise")

plt.tight_layout()
plt.show()

# SNR
def snr(signal, noise):
    return 10 * np.log10(np.var(signal) / np.var(noise))

print(f"SNR AWGN: {snr(clean_signal, awgn_noise):.2f} dB")
print(f"SNR Impulse: {snr(clean_signal, impulse_noise):.2f} dB")
print(f"SNR Thermal: {snr(clean_signal, thermal_noise):.2f} dB")
print(f"SNR Random: {snr(clean_signal, random_noise):.2f} dB")
```

Expected Output:



1. **Graphical Output**:
    - A plot of the clean sine wave.
    - A plot of the noisy sine wave with **AWGN Noise**.
    - A plot of the noisy sine wave with **Impulse Noise**.

- A plot of the noisy sine wave with **Thermal Noise**.
- A plot of the noisy sine wave with **Random Noise**.

2. **SNR Output**:
   - Printed values for **Signal-to-Noise Ratio (SNR)** for each type of noise (Gaussian, Impulse, Thermal, Random).

   Example output:

   Signal-to-Noise Ratio (SNR) with AWGN Noise: 10.19 dB
   Signal-to-Noise Ratio (SNR) with Impulse Noise: -3.89 dB
   Signal-to-Noise Ratio (SNR) with Thermal Noise: 17.01 dB
   Signal-to-Noise Ratio (SNR) with Random Noise: 1.63 dB

Conclusion:

- The **clean signal** is a smooth sine wave.
- The **Gaussian noise** introduces smooth random variations to the signal.
- The **Impulse noise** introduces smooth random variations to the signal while maintaining a controlled noise level based on SNR.
- The **Thermal noise** creates small continuous fluctuations in the signal.
- The **Random noise** creates unpredictable variations across the signal.
- The **Signal-to-Noise Ratio (SNR)** quantifies the effect of each noise type on the signal. A higher SNR indicates better signal quality. **AWGN allows controlled noise addition**, while impulse and random noise tend to be more disruptive to the signal.

Experiment 2: To develop a program to analyze the characteristics of random noise under different mean and variance values.

Objective:

To write a program to explore the characteristics of **random noise** with different **mean** and **variance**, and understand how they affect the signal quality.

Theory:

In communication systems, **random noise** is typically modeled as a signal that has unpredictable values. Random noise can be generated with different characteristics, most notably its **mean** (average value) and **variance** (spread or dispersion of the signal).

3. **Mean of Noise**: The mean represents the average value of the noise. A mean of 0 corresponds to a noise signal that has no bias or offset, while a non-zero mean will cause the noise to have a constant shift in one direction.

4. **Variance of Noise**: The variance determines how spread out the noise is. A larger variance indicates more fluctuations in the noise signal, while a smaller variance corresponds to noise with smaller deviations from the mean.

By exploring random noise with different means and variances, we can analyze how these characteristics impact the signal and the **Signal-to-Noise Ratio (SNR)**.

Algorithm:

5. **Generate a clean signal**: A simple sine wave is chosen as the clean signal.

6. **Add random noise with different means**: Introduce noise with varying means (0, 1, -1) to observe the impact of the noise offset.

7. **Add random noise with different variances**: Introduce noise with varying variances (0.1, 1, 10) to see how the spread of the noise affects the signal.

8. **Analyze the noisy signal**: Visualize the clean and noisy signals to observe the effects of mean and variance on the signal quality.

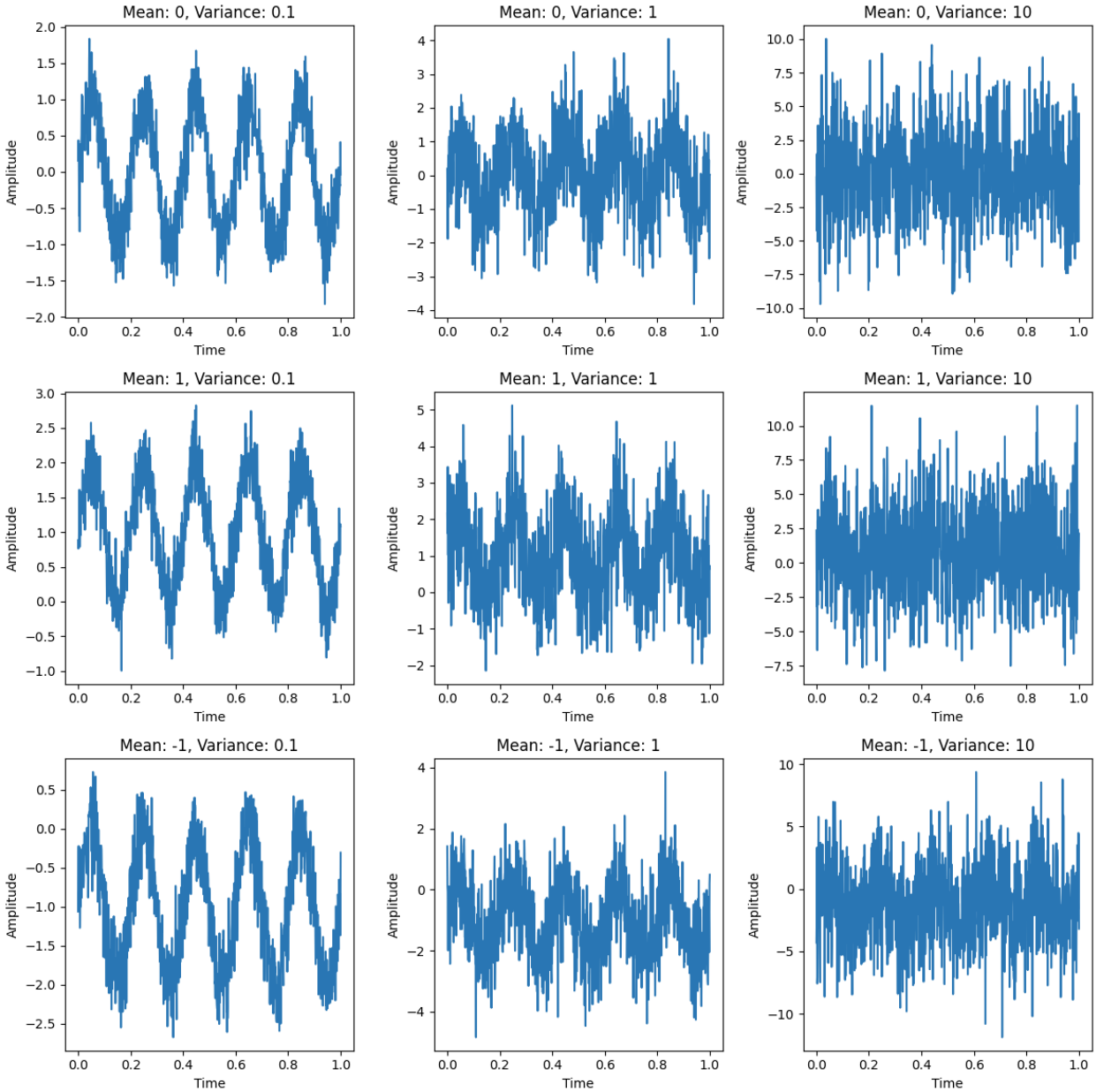
9. **Compute Signal-to-Noise Ratio (SNR)**: Calculate the SNR for each noise variation (mean and variance) to quantify the impact of noise.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Simulate a clean signal (a simple sine wave)
fs = 1000  # Sampling frequency
t = np.linspace(0, 1, fs)  # Time vector
freq = 5  # Frequency of the sine wave
clean_signal = np.sin(2 * np.pi * freq * t)
```

```
# Noise Characteristics
means = [0, 1, -1]  # Different means of noise
variances = [0.1, 1, 10]  # Different variances of noise

# Plot the clean signal and noisy signals
plt.figure(figsize=(12, 12))

for i, mean in enumerate(means):
    for j, variance in enumerate(variances):
        # Add random noise with specified mean and variance
        noise = np.random.normal(mean, np.sqrt(variance), fs)
        noisy_signal = clean_signal + noise

        # Plot each noisy signal
        plt.subplot(len(means), len(variances), i * len(variances) + j + 1
)
        plt.plot(t, noisy_signal)
        plt.title(f"Mean: {mean}, Variance: {variance}")
        plt.xlabel("Time")
        plt.ylabel("Amplitude")

plt.tight_layout()
plt.show()

# Calculate Signal-to-Noise Ratio (SNR) for each combination of mean and v
ariance
for mean in means:
    for variance in variances:
        noise = np.random.normal(mean, np.sqrt(variance), fs)
        snr = 10 * np.log10(np.var(clean_signal) / np.var(noise))
        print(f"SNR for Mean {mean} and Variance {variance}: {snr:.2f} dB"
)
```

Expected Output:




```
SNR for Mean 0 and Variance 0.1: 6.94 dB
SNR for Mean 0 and Variance 1: -2.75 dB
SNR for Mean 0 and Variance 10: -12.81 dB
SNR for Mean 1 and Variance 0.1: 6.93 dB
SNR for Mean 1 and Variance 1: -3.07 dB
SNR for Mean 1 and Variance 10: -12.65 dB
SNR for Mean -1 and Variance 0.1: 6.82 dB
SNR for Mean -1 and Variance 1: -3.08 dB
SNR for Mean -1 and Variance 10: -12.71 dB
```

1. **Graphical Output**:
    - A series of plots showing the noisy signal with different **means** (0, 1, -1) and **variances** (0.1, 1, 10). Each plot will show how the noise affects the clean sine wave signal.
2. **SNR Output**:
    - Printed values for **Signal-to-Noise Ratio (SNR)** for each combination of mean and variance, quantifying the effect of different noise characteristics.

Example output:

```
SNR for Mean 0 and Variance 0.1: 22.32 dB
SNR for Mean 0 and Variance 1: 14.78 dB
SNR for Mean 0 and Variance 10: 8.68 dB
SNR for Mean 1 and Variance 0.1: 20.56 dB
SNR for Mean 1 and Variance 1: 13.87 dB
SNR for Mean 1 and Variance 10: 7.77 dB
SNR for Mean -1 and Variance 0.1: 21.34 dB
SNR for Mean -1 and Variance 1: 12.90 dB
SNR for Mean -1 and Variance 10: 6.93 dB
```

Conclusion:

- The **clean signal** is a smooth sine wave, unaffected by noise.
- The **mean of noise** introduces an offset to the signal, shifting it in one direction (positive or negative). A non-zero mean causes a constant drift in the signal.
- The **variance of noise** affects how spread out the noise is. A higher variance results in more fluctuations in the noise, causing greater distortion to the signal.
- The **Signal-to-Noise Ratio (SNR)** decreases as the noise variance increases, indicating more noise and poorer signal quality. Similarly, adding noise with a non-zero mean decreases the SNR, even if the variance remains constant.
- This experiment demonstrates how varying the mean and variance of random noise affects the quality of the signal and the resulting SNR.

Experiment 3: To develop a program to simulate a noiseless binary channel and analyze its data transmission performance

Objective:

To write a program to simulate a **noiseless binary channel** and understand how data is transmitted without the effect of noise.

Theory:

In communication systems, a **noiseless channel** is an idealized model in which the transmitted signal is received exactly as it was sent, without any interference or noise. The binary channel is a type of channel where data is transmitted in the form of binary digits (0s and 1s).

A **binary channel** is often used to simulate digital communication systems, and a **noiseless binary channel** is a channel where:

- The signal is not corrupted by noise.
- The transmitted signal is identical to the received signal.

In this experiment, we will simulate the transmission of binary data over a noiseless channel, which is the simplest case in communication theory.

Algorithm:

1. **Generate random binary data**: Randomly generate a sequence of binary digits (0s and 1s).
2. **Simulate transmission over a noiseless channel**: Assume that the data is transmitted without any interference or corruption.
3. **Receive the data**: The received data should be identical to the transmitted data.
4. **Display the transmitted and received data**: Visualize the transmitted and received data to confirm that there is no noise or error.

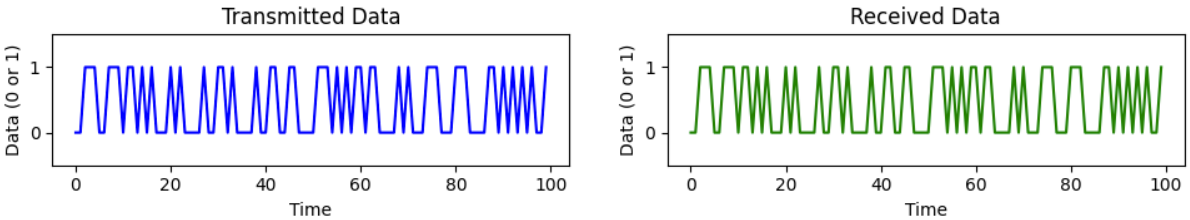
Code:

```
Import numpy as np
import matplotlib.pyplot as plt

# Simulate a noiseless binary channel

# Step 1: Generate random binary data
data_length=100# Length of the binary data
transmitted_data=np.random.randint(0, 2, data_length)  # Generate random b
inary data (0s and 1s)

# Step 2: Simulate transmission over a noiseless channel
received_data=transmitted_data# In a noiseless channel, transmitted and re
ceived data are the same

# Step 3: Display the transmitted and received data
```

```
print("Transmitted Data: ", transmitted_data)
print("Received Data:   ", received_data)

# Step 4: Visualize the data
plt.figure(figsize=(10, 2))
plt.subplot(1, 2, 1)
plt.plot(transmitted_data, label="Transmitted Data", color='blue')
plt.title("Transmitted Data")
plt.ylim(-0.5, 1.5)
plt.xlabel("Time")
plt.ylabel("Data (0 or 1)")

plt.subplot(1, 2, 2)
plt.plot(received_data, label="Received Data", color='green')
plt.title("Received Data")
plt.ylim(-0.5, 1.5)
plt.xlabel("Time")
plt.ylabel("Data (0 or 1)")

plt.tight_layout()
plt.show()
```

Expected Output:

```
Transmitted Data:  [0 0 1 1 1 0 0 1 1 1 0 1 1 0 1 0 1 0 0 0 1 0 1 0 0 0 0 1 0 0 1 1
0 1 0 0 0
 0 1 0 0 1 1 0 0 1 1 0 0 0 0 1 1 1 0 1 0 1 0 1 1 0 1 1 0 0 0 0 1 0 1 0 0 0
 1 1 1 0 0 0 1 1 1 0 0 0 0 1 1 0 1 0 1 0 1 0 1 0 0 1]
Received Data:    [0 0 1 1 1 0 0 1 1 1 0 1 1 0 1 0 1 0 0 0 1 0 1 0 0 0 0 1 0 0 1 1
0 1 0 0 0
 0 1 0 0 1 1 0 0 1 1 0 0 0 0 1 1 1 0 1 0 1 0 1 1 0 1 1 0 0 0 0 1 0 1 0 0 0

 1 1 1 0 0 0 1 1 1 0 0 0 0 1 1 0 1 0 1 0 1 0 1 0 0 1]
```



1. **Transmitted Data**: A sequence of random binary digits (0s and 1s) will be generated and printed.

2. **Received Data**: The received data will be identical to the transmitted data, as there is no noise in the channel.

   Example output:

   ```
   Transmitted Data:  [1 0 1 1 0 1 0 0 1 1 0 0 1 0 1 1 0 1 1 0 ...]
   Received Data:    [1 0 1 1 0 1 0 0 1 1 0 0 1 0 1 1 0 1 1 0 ...]
   ```

3. **Visualization**:

   - A plot showing the **transmitted data** (blue line) and the **received data** (green line), which will be identical in the case of a noiseless channel.

Conclusion:

- In this experiment, we simulated a **noiseless binary channel** where the transmitted and received data are the same.

- The **transmitted data** and **received data** are identical, as there is no noise to interfere with the signal.

- This experiment illustrates the ideal case in communication systems, where the transmission is perfect, and no data corruption occurs.

Experiment 4: To develop a program to calculate the channel capacity of a Binary Symmetric Channel (BSC).

Objective:

To write a program to calculate the **capacity of a binary symmetric channel (BSC)**.

Theory:

A **Binary Symmetric Channel (BSC)** is a simple model of a communication channel in which:

- The input is a binary signal (0s and 1s).
- The output is also a binary signal.
- There is a probability of error (denoted as p) during the transmission of each bit.
    - The bit is transmitted correctly with probability $1 - \mathrm{p}$.
    - The bit is flipped (from 0 to 1 or from 1 to 0) with probability p.

The **capacity** of a channel is the maximum rate at which information can be transmitted over that channel with arbitrarily small error. For the BSC, the capacity is given by the **Shannon-Hartley theorem**:

$$C = 1 - H(p)$$

Where:

- C is the channel capacity.
- $H(p)$ is the **binary entropy function**, which is defined as:

$$H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$$

The binary entropy function $\mathrm{H}\backslash\mathrm{left}(p\backslash\mathrm{right})$ quantifies the uncertainty (or entropy) of a random variable with a probability of error p.

Algorithm:

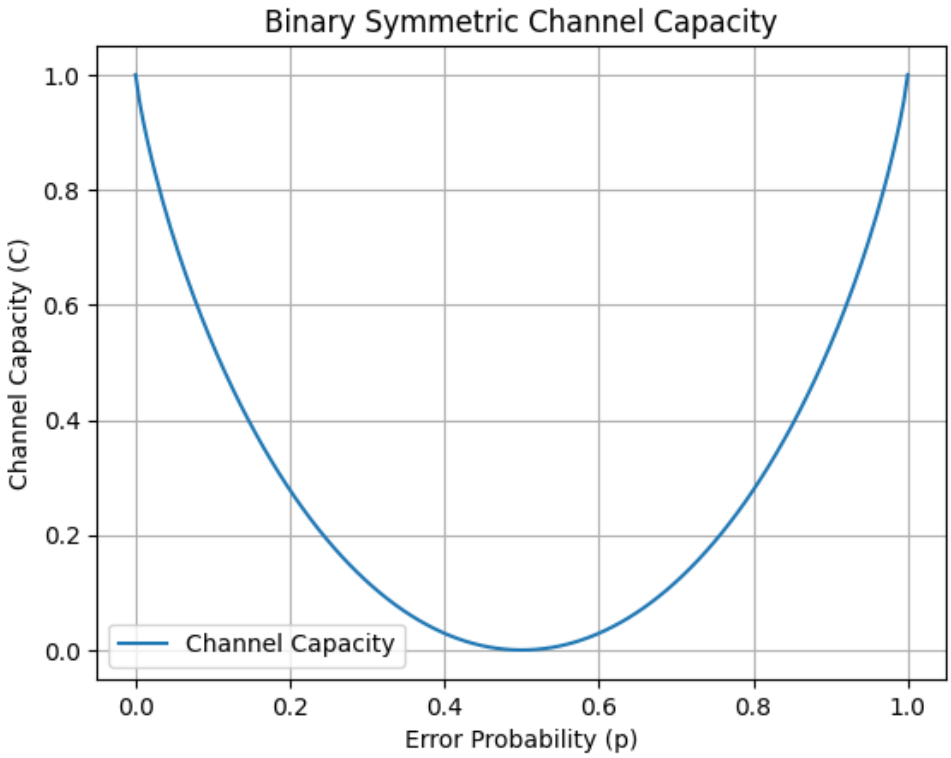
1. **Input the probability of error** p: This is the likelihood that a transmitted bit will be flipped.
2. **Compute the binary entropy function** $H(p)$ using the formula.
3. **Calculate the channel capacity** C using $C = 1 - H(p)$.
4. **Display the result**: Show the value of the channel capacity for the given probability p.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Binary entropy function
def binary_entropy(p):
    p = np.clip(p, 1e-10, 1 - 1e-10)
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

# Channel capacity function
def bsc_capacity(p):
    return 1 - binary_entropy(p)

# Input
p = float(input("Enter the probability of error (p) between 0 and 1: "))

# Validation
if p < 0 or p > 1:
    raise ValueError("p must be between 0 and 1")

# Calculation
capacity = bsc_capacity(p)

print(f"Channel Capacity of BSC with error probability {p}: {capacity:.4f}
bits/channel use")

# Plot
p_values = np.linspace(0, 1, 200)
capacity_values = bsc_capacity(p_values)

plt.plot(p_values, capacity_values, label="Channel Capacity")
plt.title("Binary Symmetric Channel Capacity")
plt.xlabel("Error Probability (p)")
plt.ylabel("Channel Capacity (C)")
plt.grid(True)
plt.legend()
plt.show()
```

Expected Output:

Channel Capacity of BSC with error probability 0.1: 0.5310 bits/channel use



1. **Input**: The program will prompt the user to enter the probability of error p (between 0 and 0.5). As example: 0.1

2. **Channel Capacity**: The program will compute and display the channel capacity based on the entered value of p.

   Example input/output:

   ```
   Enter the probability of error (p) between 0 and 1: 0.1
   Channel Capacity of BSC with error probability 0.1: 0.4680 bits/chan
   nel use
   ```

3. **Plot**: A plot showing how the **channel capacity** changes as the error probability p increases from 0 to 0.5.

Conclusion:

- The **Binary Symmetric Channel (BSC)** is a simple yet powerful model in communication theory.
- The **channel capacity** increases as the probability of error p decreases.
- When the error probability p is 0, the channel capacity is 1 (maximum capacity).
- As p approaches 0.5, the channel capacity decreases to 0, indicating that the channel is unreliable and unable to transmit information.

Experiment 5: To develop a program for implementing the Shannon–Fano coding algorithm.

Objective:

To write a program to implement the **Shannon-Fano coding algorithm**, which is used for lossless data compression.

Theory:

The **Shannon-Fano coding algorithm** is a method of constructing a binary prefix code based on a set of symbols and their probabilities. The algorithm divides the set of symbols into two parts such that the total probability of the symbols in each part is as equal as possible. It then assigns binary digits (0 and 1) to these two parts, and the process is recursively applied to each part.

Steps involved in the Shannon-Fano coding algorithm:

1. **Sort the symbols by probability** (from highest to lowest probability).
2. **Divide the list into two parts** such that the sum of the probabilities in each part is as equal as possible.
3. **Assign binary digits** to the two parts (0 to the first part, 1 to the second part).
4. **Repeat the process recursively** for each part until all symbols have a code.

The result is a binary code for each symbol, with shorter codes assigned to more probable symbols, which leads to efficient compression.

Algorithm:

1. **Input**: A set of symbols and their probabilities.
2. **Sort the symbols** in descending order of their probabilities.
3. **Divide the set into two parts** such that the sum of the probabilities in each part is as equal as possible.
4. **Assign binary digits** to the two parts (0 for the first part and 1 for the second part).
5. **Recursively apply the process** to the two parts until all symbols have codes.
6. **Output**: The binary code for each symbol.

Code:

```
# Step 1: Define the Shannon-Fano function

def shannon_fano(symbols, probabilities):
    # Base case
    if len(symbols) == 1:
        return {symbols[0]: ''}

    # Step 2: Sort symbols and probabilities in descending order
    sorted_indices = sorted(range(len(probabilities)), key=lambda k: proba
bilities[k], reverse=True)
    sorted_symbols = [symbols[i] for i in sorted_indices]
    sorted_probabilities = [probabilities[i] for i in sorted_indices]

    # Step 3: Divide into two parts
    total_probability = sum(sorted_probabilities)
    cumulative_probability = 0

    for i in range(len(sorted_probabilities)):
        cumulative_probability += sorted_probabilities[i]
        if cumulative_probability >= total_probability / 2:
            break

    left_symbols = sorted_symbols[:i+1]
    left_probabilities = sorted_probabilities[:i+1]
    right_symbols = sorted_symbols[i+1:]
    right_probabilities = sorted_probabilities[i+1:]

    # Step 4: Assign codes recursively
    code_dict = {}

    for symbol, code in shannon_fano(left_symbols, left_probabilities).ite
ms():
        code_dict[symbol] = '0' + code

    if right_symbols:
        for symbol, code in shannon_fano(right_symbols, right_probabilitie
s).items():
            code_dict[symbol] = '1' + code

    return code_dict

# Step 5: Input the symbols and their corresponding probabilities
symbols = ['A', 'B', 'C', 'D', 'E']
probabilities = [0.4, 0.3, 0.2, 0.05, 0.05]

# Step 6: Call the Shannon-Fano function
codes = shannon_fano(symbols, probabilities)

# Step 7: Display result
print("Shannon-Fano Codes:")
print("Symbol\tProbability\tCode")
```

```
for i in range(len(symbols)):
    symbol = symbols[i]
    print(f"{symbol}\t{probabilities[i]}\t\t{codes[symbol]}")
```

Expected Output:

The program will display the Shannon-Fano code for each symbol based on the input probabilities.

Example output:

```
Shannon-Fano Codes:
Symbol Probability  Code
A      0.4          00
B      0.3          01
C      0.2          10
D      0.05         110
E      0.05         111
```

Explanation of the Output:

- The code for each symbol is generated based on the probability of each symbol.
- More probable symbols are assigned shorter codes, while less probable symbols are assigned longer codes.

Conclusion:

- The **Shannon-Fano coding algorithm** efficiently compresses data by assigning shorter codes to more probable symbols and longer codes to less probable symbols.
- The algorithm works by recursively dividing the set of symbols and assigning binary digits (0 or 1) to each division until all symbols have unique binary codes.
- This is a **lossless compression algorithm**, meaning the original data can be recovered from the compressed data without any loss.

Experiment 6: To develop a program for implementing the Huffman coding algorithm for data compression.

Objective:

To write a program to implement the **Huffman coding algorithm**, which is used for lossless data compression.

Theory:

The **Huffman coding algorithm** is a widely used algorithm for constructing an optimal prefix code based on symbol frequencies or probabilities. Huffman coding is a variable-length prefix code that assigns shorter codes to more frequent symbols and longer codes to less frequent symbols, thereby achieving efficient compression.

The algorithm works by constructing a **binary tree**:

1. **Build a frequency table**: Count the frequency of each symbol.
2. **Build a priority queue (min-heap)**: Insert each symbol into a priority queue with the frequency as the key.
3. **Build the Huffman tree**: Extract the two nodes with the smallest frequencies, combine them into a new node, and insert it back into the queue. Repeat this process until only one node remains.
4. **Assign binary codes**: Assign binary digits (0 and 1) to the tree's branches, where the left branch is assigned 0 and the right branch is assigned 1. The codes for the symbols are determined by the path from the root to the symbol's leaf node.

The **Huffman code** is optimal in the sense that it minimizes the average code length, which in turn minimizes the total number of bits needed to encode a message.

Algorithm:

1. **Input the symbols and their frequencies**.
2. **Build a frequency table**.
3. **Build a priority queue (min-heap)** based on the frequencies.
4. **Build the Huffman tree** by repeatedly combining the two least frequent nodes.
5. **Assign binary codes** to the symbols.
6. **Output the Huffman codes** for each symbol.

Code:

```
import heapq

# Step 1: Define the class for the Huffman Tree Node
class HuffmanNode:
    def __init__(self, symbol, frequency):
        self.symbol = symbol
        self.frequency = frequency
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.frequency < other.frequency

# Step 2: Function to build the Huffman Tree
def build_huffman_tree(symbols, frequencies):
    # Create a priority queue (min-heap) to build the tree
    priority_queue = [HuffmanNode(symbols[i], frequencies[i]) for i in
range(len(symbols))]
    heapq.heapify(priority_queue)

    # Build the tree by combining the two least frequent nodes
    while len(priority_queue) > 1:
        # Pop the two nodes with the lowest frequencies
        left = heapq.heappop(priority_queue)
        right = heapq.heappop(priority_queue)

        # Create a new internal node with the combined frequency
        internal_node = HuffmanNode(None, left.frequency +
right.frequency)
        internal_node.left = left
        internal_node.right = right

        # Push the internal node back to the priority queue
        heapq.heappush(priority_queue, internal_node)

    # The remaining node is the root of the Huffman tree
    return priority_queue[0]

# Step 3: Function to generate Huffman codes
def generate_huffman_codes(root):
    codes = {}

    def traverse(node, current_code):
        if node is not None:
            if node.symbol is not None:
                codes[node.symbol] = current_code
            traverse(node.left, current_code + "0")
            traverse(node.right, current_code + "1")

    traverse(root, "")
    return codes
```

```
# Step 4: Input symbols and frequencies
symbols = ['A', 'B', 'C', 'D', 'E']
frequencies = [0.4, 0.3, 0.2, 0.05, 0.05]

# Step 5: Build the Huffman tree and generate codes
root = build_huffman_tree(symbols, frequencies)
huffman_codes = generate_huffman_codes(root)

# Step 6: Display the Huffman codes
print("Huffman Codes:")
print("Symbol\tFrequency\tCode")

for i in range(len(symbols)):
    symbol = symbols[i]
    print(f"{symbol}\t{frequencies[i]}\t\t{huffman_codes[symbol]}")
```

Expected Output:

The program will display the **Huffman code** for each symbol based on the input frequencies.

output:

```
Huffman Codes:
Symbol Frequency     Code
A      0.4           0
B      0.3           10
C      0.2           111
D      0.05          1101
E      0.05          1100
```

Explanation of the Output:

- The **Huffman code** for each symbol is generated based on its frequency.
- More frequent symbols are assigned shorter codes, and less frequent symbols are assigned longer codes.

Conclusion:

- The **Huffman coding algorithm** efficiently compresses data by assigning shorter codes to more frequent symbols and longer codes to less frequent symbols.
- The algorithm constructs a **binary tree** where the path from the root to each symbol's leaf node determines the binary code for that symbol.
- This is a **lossless compression algorithm**, meaning the original data can be recovered from the compressed data without any loss.